

GrowthBot DE

Project summary covering product intent, current architecture, engineering constraints, and forward roadmap.

1. Project

Name	GrowthBot DE
Type	AI-assisted sales platform with Telegram conversational endpoint and Next.js operational dash
Primary market	DACH, with German-first bot interactions and product recommendations
Current shape	Working local prototype with conversational orchestration, follow-up automation, affiliate click tra

2. Project Context

GrowthBot DE was conceived as more than a chatbot experiment. The project is intended to validate how a language-model-driven assistant can support product recommendation, user re-engagement, and operational monitoring inside a lightweight sales workflow.

The current implementation combines a Telegram bot for lead interaction, an LLM-backed recommendation pipeline, SQLite persistence for local state, and a Next.js dashboard for tracking sessions, clicks, and follow-up activity.

3. Clear Objective

The clear product objective is to convert conversational user intent into a traceable recommendation flow: detect interest, map relevant products, generate a persuasive response, store the interaction, schedule re-engagement, and expose the operational outcome to an administrator.

- Give the user a guided Telegram experience with category shortcuts and free-text input.
- Use structured LLM output to recommend exactly one product from a curated catalog.
- Persist sessions and schedule follow-ups automatically.
- Track affiliate clicks and expose core metrics through an admin surface.

4. Objectives

- Build a demonstrable AI sales assistant that is simple enough to run locally and clear enough to explain to recruiters, collaborators, or stakeholders.
- Separate conversational orchestration from dashboard and tracking concerns so the architecture can evolve beyond a prototype.
- Create a foundation for future migration from local-only persistence to a shared live environment.
- Keep the system understandable: no unnecessary abstractions, clear module boundaries, and straightforward control flow.

5. Stacks

Frontend / Admin	Next.js 15.5.2, React 19.1.1, React DOM 19.1.1
Language / Build	TypeScript 5.9.3, tsx 4.20.5
Bot channel	node-telegram-bot-api 0.67.0
LLM layer	OpenAI SDK 6.35.0
Validation	zod 4.1.5
Persistence	SQLite via better-sqlite3 12.9.0
Config	dotenv 17.4.2
Tests	Vitest 3.2.4

6. Structure

- src/integrations/telegram/: Telegram adapter handling messages, callbacks, and user session reset commands.
- src/modules/conversation/: Core orchestrator that loads history, classifies intent, matches products, builds prompts, calls OpenAI, and persists outcomes.
- src/modules/intent/: Keyword-based intent classification.
- src/modules/products/: Product matching logic based on tags and rank scoring.
- src/llm/: Prompt builder plus structured response generation through OpenAI.
- src/modules/followup/: Scheduled follow-up sender that builds tracked redirect links.
- src/database/: SQLite client and repository functions for sessions, clicks, and follow-ups.
- src/app/: Next.js routes for dashboard, health endpoint, and click redirect tracking.
- src/catalog/: Static product catalog data and access helpers.

7. Current Vision

The project currently behaves like a strong local prototype and portfolio-grade architecture case. The system already demonstrates a full loop from chat entry to recommendation, persistence, follow-up scheduling, redirect tracking, and dashboard visibility.

At the same time, the current implementation is not yet a production-grade live platform. The main limiting factor is the local SQLite database, which does not support an easy split between separately deployed web and bot runtimes without additional infrastructure changes.

8. Current Strengths

- Clear module boundaries around bot handling, orchestration, LLM use, persistence, and dashboard concerns.
- A demonstrable operator surface instead of a hidden backend-only prototype.
- Structured recommendation generation rather than raw free-form chat completion output.
- Practical product flow: interest -> recommendation -> persistence -> follow-up -> click tracking.
- Good portfolio narrative: transition from simple bot logic to operational AI platform architecture.

9. Current Constraints

- SQLite is local-only and becomes the core blocker for a true multi-service live deployment.
- Intent classification is keyword-driven and can miss nuance or mixed intent messages.
- No production-ready auth, access control, audit trail, or environment segregation for operator usage.
- The testing setup exists, but environment and config resolution still need cleanup for reliable execution in all contexts.
- Polling-based bot runtime is practical for local use but not the ideal final runtime model for a public deployment.

10. Next Sprints

- Sprint 1 - Production-readiness baseline: rotate secrets, clean environment handling, define .env.example, and document safe local startup.

- Sprint 2 - Shared persistence migration: replace SQLite with a shared database such as Postgres or Turso so the bot and web app can run separately.
- Sprint 3 - Deployable demo: publish the Next.js dashboard and set up a stable bot runtime with shared persistence.
- Sprint 4 - Recommendation quality: improve intent detection, catalog matching, fallback behavior, and structured validation around LLM output.
- Sprint 5 - Recruiter-facing polish: better README, short architecture summary, screenshots, and a public demo path that works without manual intervention.

11. Future Vision

The longer-term vision is a deployable AI sales platform in which conversational entry, recommendation logic, persistence, follow-up automation, click measurement, and operator insights work as one shared system.

- Shared remote database backing all runtimes.
- Stable production deployment with web and bot components.
- Better intent detection and recommendation confidence logic.
- Cleaner analytics around clicks, follow-up success, and conversion signals.
- Potential support for broader channels beyond Telegram if the core architecture remains stable.

12. Engineering Practices To Avoid

The project benefits from discipline more than from added complexity. The following practices should be explicitly avoided when coding, creating components, or changing structure.

- Do not introduce abstractions before there is repeated pressure for them. Premature indirection would make a currently understandable system harder to reason about.
- Do not mix UI concerns, bot handling, and orchestration logic inside the same module. Boundaries are one of the strongest current assets of the project.
- Do not add components or routes without a real operator or user need. Every new surface should support a traceable product workflow.
- Do not couple storage assumptions to local-only runtime forever. Any new data flow should keep shared-persistence migration in mind.
- Do not rely on undocumented environment variables, hidden setup steps, or manual operational knowledge.
- Do not let LLM responses bypass structured checks when the application expects a single valid product recommendation.
- Do not create visual complexity in the dashboard without improving operational comprehension.
- Do not expose secrets in repository files, screenshots, or deploy logs.

13. Practical Rules For Future Changes

- Read the existing flow before editing: Telegram adapter -> orchestrator -> repositories -> dashboard/redirect routes.
- Preserve simple data flow. New logic should be added where responsibility already exists, not in a convenient random file.
- When adding a component, ask whether it clarifies the operator view or just adds presentation noise.
- When changing storage, design for migration safety and explicit repository boundaries.
- When changing LLM behavior, keep prompts, selection logic, and output validation aligned.
- When adding tests, prefer business-critical behavior first: intent mapping, product selection, persistence effects, and redirect tracking.

14. Recommended Immediate Actions

- Rotate exposed Telegram and OpenAI credentials immediately.

- Add a public-facing README that explains architecture, local startup, and demo limitations clearly.
- Prepare a minimal live demo path before presenting the project as publicly accessible.
- Prioritize shared persistence before claiming the bot and dashboard are fully live together.

Summary note: GrowthBot DE is already strong as an architecture and product-engineering case. Its next step is not more surface area, but stronger deployment realism and operational reliability.